Multi-Task Surrogate Models for Mechanical MNIST Cahn–Hilliard

Model Architecture and Training Documentation

AISA Hackathon

March 27, 2026

Abstract

This document provides a detailed description of two neural network surrogate models developed for the Mechanical MNIST Cahn–Hilliard dataset: a **Multi-Task U-Net** (∼8.1M parameters) and a **Multi-Task Fourier Neural Operator** (∼2.7M parameters). Both models simultaneously predict displacement fields, strain energy, and reaction forces from binary microstructure images, replacing expensive finite element simulations with fast neural network inference.

Contents

1 Problem Description **2**
1.1 Dataset Overview . . . 2
1.2 Prediction Targets . . . 2

2 Data Preprocessing **2**
2.1 Image Downsampling . . . 2
2.2 Output Normalization . . . 2
2.3 Data Split . . . 3

3 Model 1: Multi-Task U-Net **3**
3.1 Architecture Overview . . . 3
3.2 ConvBlock . . . 4
3.3 Encoder . . . 4
3.4 Displacement Decoder (Head 1) . . . 4
3.5 Strain Energy Head (Head 2) . . . 4
3.6 Reaction Forces Head (Head 3) . . . 4

4 Model 2: Multi-Task Fourier Neural Operator **4**
4.1 Architecture Overview . . . 4
4.2 Spectral Convolution . . . 5
4.3 FNO Block . . . 5
4.4 Lifting and Projection . . . 6
4.5 Output Heads . . . 6

5 Training Process **6**
5.1 Loss Function . . . 6
5.2 Hyperparameters . . . 7
5.3 Learning Rate Schedule . . . 7
5.4 Early Stopping . . . 7
5.5 Training Algorithm . . . 7

6 Evaluation **8**
6.1 Metrics 8
6.2 Denormalization 8

7 Model Comparison **8**
7.1 Theoretical Considerations 8

8 FNO-Specific Hyperparameters **9**

9 Implementation Details **9**
9.1 Data Loading 9
9.2 Checkpoint Format 9
9.3 Inference Pipeline 10

1 Problem Description

1.1 Dataset Overview

The **Mechanical MNIST Cahn–Hilliard** dataset contains 104,813 samples of binary heterogeneous microstructures generated via the Cahn–Hilliard equation. Each sample is paired with finite element analysis (FEA) simulation results computed using FEniCS.

Table 1: Dataset composition.

Case	Samples	Image IDs
Case 1	37,523	1 – 37,523
Case 2	37,680	37,524 – 75,203
Case 3	29,610	75,204 – 104,813
Total	**104,813**	

The physical simulation applies **equibiaxial loading** to a binary Neo-Hookean material at seven displacement levels:

$$d \in \{0.0,\ 0.001,\ 0.1,\ 0.2,\ 0.3,\ 0.4,\ 0.5\}.$$

1.2 Prediction Targets

Both models are trained as **multi-task** learners with three output heads:

1. **Displacement fields** – Two spatial fields $u_x, u_y \in \mathbb{R}^{64\times 64}$ representing the x- and y-direction displacements at $d = 0.5$. Output shape: $(B, 2, 64, 64)$.
2. **Strain energy** – Seven scalar values, one per displacement level. Output shape: $(B, 7)$.
3. **Reaction forces** – Twenty-eight scalar values (4 boundaries $\times$ 7 displacement levels). Boundaries: left-x, right-x, bottom-y, top-y. Output shape: $(B, 28)$.

2 Data Preprocessing

2.1 Image Downsampling

The raw microstructure images are 400×400 binary matrices. They are downsampled to 64×64 using the following pipeline:

1. **Center crop** – Remove 8-pixel border on each side to obtain a 384×384 region.
2. **Block averaging** – Divide into 6×6 blocks and compute the mean of each block, yielding a 64×64 image.

Displacement fields undergo the same spatial downsampling.

2.2 Output Normalization

All output targets are normalized using per-channel z-score normalization:

$$\hat{y} = \frac{y - \mu}{\sigma + \epsilon}, \qquad \epsilon = 10^{-10},$$

where μ and σ are the training-set mean and standard deviation. Separate statistics are computed for:

- u_x: scalar mean/std over all pixels and samples.
- u_y: scalar mean/std over all pixels and samples.
- Strain energy: element-wise mean/std vectors of length 7.
- Reaction forces: element-wise mean/std vectors of length 28.

The statistics are stored in `norm_stats.npy` and used for denormalization during evaluation.

2.3 Data Split

The full dataset is split with a fixed random seed (seed = 42):

Table 2: Train / validation / test split.

Split	Fraction	Samples
Training	80%	83,850
Validation	10%	10,481
Test	10%	10,482

3 Model 1: Multi-Task U-Net

3.1 Architecture Overview

The Multi-Task U-Net follows the classical encoder–decoder structure with skip connections, extended with two additional fully connected heads for scalar prediction tasks. The total parameter count is approximately **8.1 million**.

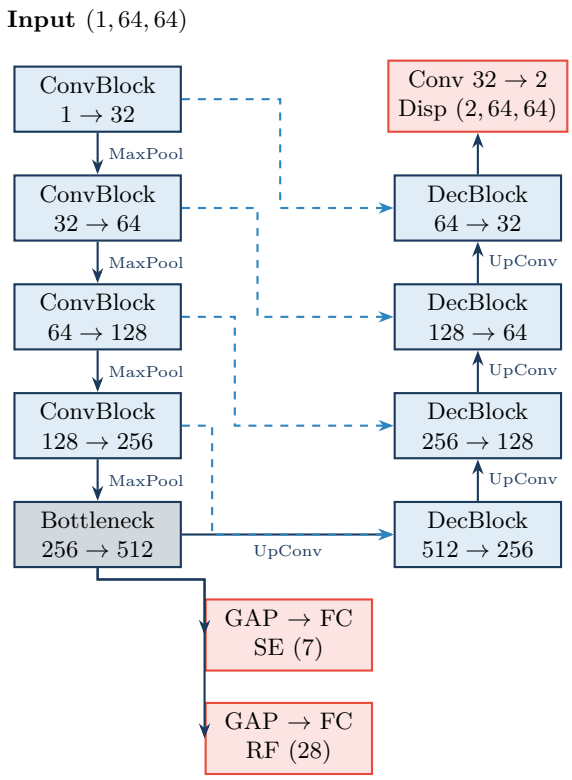

Figure 1: Multi-Task U-Net architecture. Dashed arrows denote skip connections. GAP = Global Average Pooling, FC = Fully Connected layers.

3.2 ConvBlock

Each `ConvBlock` consists of two consecutive convolution layers:

$$\text{Conv2d}(3{\times}3) \;\rightarrow\; \text{BN} \;\rightarrow\; \text{ReLU} \;\rightarrow\; \text{Conv2d}(3{\times}3) \;\rightarrow\; \text{BN} \;\rightarrow\; \text{ReLU}$$

All convolutions use padding = 1 (preserving spatial dimensions) and bias = False (since batch normalization follows).

3.3 Encoder

The shared encoder has four stages with feature dimensions $[32, 64, 128, 256]$. Each stage applies a `ConvBlock` followed by 2×2 max-pooling, progressively reducing the spatial resolution:

$$64^2 \xrightarrow{\text{pool}} 32^2 \xrightarrow{\text{pool}} 16^2 \xrightarrow{\text{pool}} 8^2 \xrightarrow{\text{pool}} 4^2.$$

A bottleneck `ConvBlock` maps $256 \rightarrow 512$ channels at 4×4 spatial resolution.

3.4 Displacement Decoder (Head 1)

The decoder mirrors the encoder with four upsampling stages using transposed convolutions (`ConvTranspose2d`, kernel 2, stride 2). At each level, the upsampled features are concatenated with the corresponding encoder skip connection and refined through a `ConvBlock`:

$$512 \;\rightarrow\; 256 \;\rightarrow\; 128 \;\rightarrow\; 64 \;\rightarrow\; 32.$$

A final 1×1 convolution projects the 32-channel feature map to 2 channels, producing the displacement output $(B, 2, 64, 64)$.

3.5 Strain Energy Head (Head 2)

From the bottleneck representation $(B, 512, 4, 4)$:

$$\text{AdaptiveAvgPool}(1) \;\rightarrow\; \text{Flatten} \;\rightarrow\; \text{FC}(512, 128) \;\rightarrow\; \text{ReLU} \;\rightarrow\; \text{FC}(128, 7).$$

Output: $(B, 7)$ – strain energy at each of the seven displacement levels.

3.6 Reaction Forces Head (Head 3)

Identical structure to the strain energy head, with a different output dimension:

$$\text{AdaptiveAvgPool}(1) \;\rightarrow\; \text{Flatten} \;\rightarrow\; \text{FC}(512, 128) \;\rightarrow\; \text{ReLU} \;\rightarrow\; \text{FC}(128, 28).$$

Output: $(B, 28)$ – reaction forces (4 boundaries $\times$ 7 levels).

4 Model 2: Multi-Task Fourier Neural Operator

4.1 Architecture Overview

The Multi-Task FNO learns solution operators in Fourier space through spectral convolutions. It maintains **full spatial resolution** (64×64) throughout the network – no pooling or upsampling is needed for the displacement head. The total parameter count is approximately **2.7 million**.

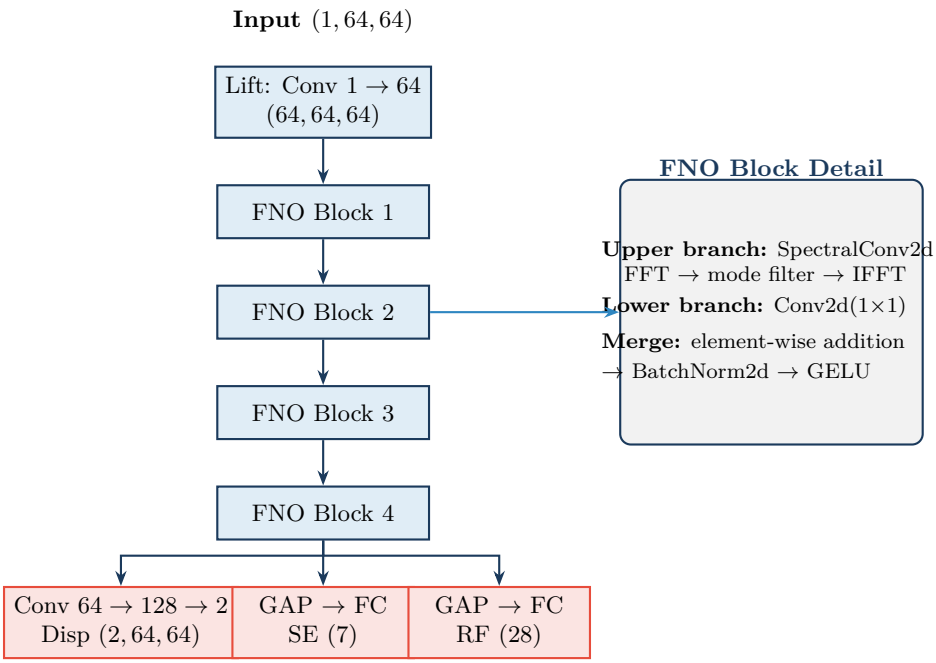

Figure 2: Multi-Task FNO architecture with FNO block detail.

4.2 Spectral Convolution

The core operation is the **SpectralConv2d** layer, which learns weights directly in the Fourier domain:

1. Apply 2D real FFT to the input: $\hat{x} = \mathrm{rfft2}(x)$.
2. Retain only the lowest $M_1 \times M_2$ Fourier modes (default: $M_1 = M_2 = 16$).
3. Multiply selected modes with learnable complex weights via Einstein summation:

$$\hat{y}_{b,o,k_1,k_2} = \sum_i \hat{x}_{b,i,k_1,k_2} \cdot W_{i,o,k_1,k_2}, \qquad W \in \mathbb{C}^{C_{\text{in}} \times C_{\text{out}} \times M_1 \times M_2}.$$

   Two weight tensors handle positive and negative frequency corners.

4. Apply inverse FFT: $y = \mathrm{irfft2}(\hat{y})$.

Weights are initialized with scale $1/(C_{\text{in}} \cdot C_{\text{out}})$.

4.3 FNO Block

Each FNO block combines spectral and pointwise convolution paths:

$$\text{out} = \mathrm{GELU}\Big(\mathrm{BN}\big(\underbrace{\mathrm{SpectralConv2d}(x)}_{\text{global (Fourier)}} + \underbrace{\mathrm{Conv2d}_{1\times1}(x)}_{\text{local (pointwise)}}\big)\Big).$$

The spectral branch captures global spatial patterns through frequency-domain filtering, while the pointwise branch processes channel interactions locally. Four such blocks are stacked in sequence, all operating at full 64×64 resolution.

4.4 Lifting and Projection

- **Lift**: A 1×1 convolution maps the single-channel input to $W = 64$ channels: $(B, 1, 64, 64) \to (B, 64, 64, 64)$.
- **Projection**: After the FNO layers, each head projects the latent features to its respective output.

4.5 Output Heads

Displacement Head. Two 1×1 convolutions with GELU activation:

$$\text{Conv}(64, 128) \;\to\; \text{GELU} \;\to\; \text{Conv}(128, 2).$$

Output: $(B, 2, 64, 64)$.

Strain Energy Head.

$$\text{AdaptiveAvgPool}(1) \;\to\; \text{Flatten} \;\to\; \text{FC}(64, 128) \;\to\; \text{ReLU} \;\to\; \text{FC}(128, 7).$$

Output: $(B, 7)$.

Reaction Forces Head.

$$\text{AdaptiveAvgPool}(1) \;\to\; \text{Flatten} \;\to\; \text{FC}(64, 128) \;\to\; \text{ReLU} \;\to\; \text{FC}(128, 28).$$

Output: $(B, 28)$.

5 Training Process

Both models share an identical training pipeline, enabling fair comparison.

5.1 Loss Function

The total loss is an **unweighted sum** of MSE losses across the three tasks:

$$\mathcal{L}_{\text{total}} = \underbrace{\frac{1}{N}\sum_i \|\hat{u}_i - u_i\|_2^2}_{\mathcal{L}_{\text{disp}}} + \underbrace{\frac{1}{N}\sum_i \|\hat{s}_i - s_i\|_2^2}_{\mathcal{L}_{\text{SE}}} + \underbrace{\frac{1}{N}\sum_i \|\hat{r}_i - r_i\|_2^2}_{\mathcal{L}_{\text{RF}}},$$

where N is the batch size, $\hat{\cdot}$ denotes predictions, and u, s, r are the displacement, strain energy, and reaction force targets, respectively.

5.2 Hyperparameters

Table 3: Training hyperparameters (shared by both models).

Parameter	**Value**
Optimizer	Adam
Learning rate	1×10^{-3}
Weight decay	1×10^{-5}
Batch size	64
Maximum epochs	50
LR scheduler	ReduceLROnPlateau
Reduction factor	0.5
Scheduler patience	3 epochs
Early stopping patience	7 epochs
Random seed	42

5.3 Learning Rate Schedule

The learning rate is managed by `ReduceLROnPlateau`, which monitors the total validation loss. When validation loss stops decreasing for 3 consecutive epochs, the learning rate is halved:

$$\mathrm{lr}_{\mathrm{new}} = 0.5 \times \mathrm{lr}_{\mathrm{current}}.$$

5.4 Early Stopping

Training terminates early if the total validation loss does not improve for 7 consecutive epochs. The model checkpoint with the lowest validation loss is saved as `best_model.pt`.

5.5 Training Algorithm

Algorithm 1 Multi-Task Training Loop

Require: Training set $\mathcal{D}_{\mathrm{train}}$, validation set $\mathcal{D}_{\mathrm{val}}$, model f_θ

```
Initialize θ, optimizer, scheduler
best_loss ← ∞, wait ← 0
for epoch = 1, ..., 50 do
    for each mini-batch (X, u, s, r) in D_train do
        û, ŝ, r̂ ← f_θ(X)
        L ← MSE(û, u) + MSE(ŝ, s) + MSE(r̂, r)
        Backpropagate ∇_θ L; update θ
    end for
    Compute L_val on D_val
    Update scheduler with L_val
    if L_val < best_loss then
        best_loss ← L_val; save checkpoint; wait ← 0
    else
        wait ← wait + 1
    end if
    if wait ≥ 7 then
        break                                          ▷ Early stopping
    end if
end for
```

6 Evaluation

6.1 Metrics

Models are evaluated on the held-out test set using the following metrics (computed after denormalization):

- **Displacement – Relative L^2 error**:
$$e_{\text{rel}} = \frac{\|\hat{u} - u\|_2}{\|u\|_2}$$
averaged over all test samples.
- **Displacement – MSE**: Pixel-wise mean squared error over both channels.
- **Strain energy – MSE**: Mean squared error averaged over the 7-element vectors.
- **Reaction forces – MSE**: Mean squared error averaged over the 28-element vectors.

6.2 Denormalization

Predictions are converted back to physical units before metric computation:
$$y = \hat{y} \cdot \sigma + \mu,$$
where μ and σ are the per-channel statistics stored during preprocessing.

7 Model Comparison

Table 4: Architectural comparison of the two surrogate models.

Aspect	**Multi-Task U-Net**	**Multi-Task FNO**
Backbone type	Local convolutions (3×3) with pooling/upsampling	Spectral convolutions via FFT (global receptive field)
Parameters	∼8.1M	∼2.7M
Spatial resolution	Progressive downsampling ($64 \to 4$) then upsampling	Constant 64×64 throughout
Receptive field	Local, grows with depth	Global from layer 1 (full spectrum)
Skip connections	Yes (encoder → decoder at each level)	No
Scalar head input	Bottleneck features (512 ch, 4×4)	Final FNO layer (64 ch, 64×64)
Normalization	BatchNorm (after each conv)	BatchNorm (after spectral + pointwise sum)
Activation	ReLU (throughout)	GELU (backbone) / ReLU (scalar heads)
Decoder for displacement	Transposed convolutions with skip connections	Two 1×1 convolutions (no upsampling needed)

7.1 Theoretical Considerations

- The **FNO** is theoretically well-suited for PDE surrogate modeling due to its ability to learn mappings between function spaces via spectral representations. Its global receptive field enables capturing long-range spatial correlations from the first layer.

- The **U-Net** benefits from well-established multi-scale feature extraction and skip connections that preserve fine-grained spatial details during reconstruction. Its inductive bias toward local features is effective for capturing material interfaces.
- The FNO uses $\sim 3\times$ fewer parameters, making it more computationally efficient at inference time. However, the FFT operations introduce overhead that may offset parameter savings for small input sizes.

8 FNO-Specific Hyperparameters

Table 5: FNO-specific architecture hyperparameters.

Parameter	Value	Description
Fourier modes (M)	16	Number of frequency modes retained per spatial dimension. Controls the frequency resolution of learned filters.
Width (W)	64	Channel dimension throughout FNO layers.
Number of layers	4	Number of stacked FNO blocks.
Weight dtype	`cfloat`	Complex-valued learnable weights in Fourier space.

9 Implementation Details

9.1 Data Loading

Both models use PyTorch's `DataLoader` with:

- Memory-mapped NumPy arrays (`mmap_mode="r"`) for efficient I/O on large datasets.
- Pinned memory (`pin_memory=True`) for faster GPU transfers.
- 4 parallel data loading workers.

9.2 Checkpoint Format

The saved checkpoint (`best_model.pt`) contains:

- `model_state_dict` – Trained model weights.
- `optimizer_state_dict` – Optimizer state for resuming.
- `epoch` – Epoch number of the best checkpoint.
- `val_loss` – Per-task validation losses at checkpoint.
- `train_loss` – Per-task training losses at checkpoint.
- `hparams` – Architecture hyperparameters (FNO only).

9.3 Inference Pipeline

For prediction on new images:

1. Load image (supports `.txt`, `.jpeg`, `.png`).
2. Binarize if needed (threshold at 0.5 for grayscale images).
3. Downsample to 64×64 via center crop + block averaging.
4. Normalize and run forward pass.
5. Denormalize outputs to physical units.
6. Generate FEA-style visualization with displacement heatmaps, strain energy curves, and reaction force plots.